

第四章 OpenCV 物件追蹤 (Object Tracking)

1. 上堂功課回顧 (臉部識別 Face Detection):

挑戰站：編寫一個 Python 程式《遮臉路人甲》(提交 Task3_3X99.py)

1. 結合「影片讀取」與「模糊處理」。
2. 程式必須自動識別鏡頭中的所有人脸，並立即進行 高斯模糊 (GaussianBlur)。
3. 進階要求：在畫面左上角加上文字 "Privacy Mode: ON"。

```
import cv2

# 1. 載入預訓練的人臉模型 (Haar Cascade)
face_cascade = cv2.CascadeClassifier(cv2.data.harcascades +
'haarcascade_frontalface_default.xml')
cap = cv2.VideoCapture("walkingGirl.mp4")
print("正在開啟影片... 按下 'q' 鍵可退出程式。")

while True:
    # 讀取鏡頭的每一幀 (Frame)
    ret, frame = cap.read()
    if not ret:
        print("無法讀取影片幀，請檢查設備連接。")
        break

    # 3. 轉換為灰階 (Gray Scale) 以提高識別效率
    gray = cv2.cvtColor(frame, cv2.COLOR_BGR2GRAY)

    # 4. 偵測臉部
    # scaleFactor: 縮放比例, minNeighbors: 檢測多少次才確定是臉
    faces = face_cascade.detectMultiScale(gray, scaleFactor=1.1,
minNeighbors=10, minSize=(100, 100))

    # 5. 處理偵測到的每一張臉
    for (x, y, w, h) in faces:
        # A. 擷取臉部區域 (Region of Interest, ROI)
        face_roi = frame[y:y+h, x:x+w]

        # B. 進行高斯模糊 (Gaussian Blur)
        # (99, 99) 是模糊核心大小，必須是奇數，數字越大越模糊
        blurred_face = cv2.GaussianBlur(face_roi, (99, 99), 30)

        # C. 將處理後的模糊區域放回原圖
        frame[y:y+h, x:x+w] = blurred_face

    # (選做) 可以在模糊區外面畫一個圈，提醒學生這裡被偵測到了
    cv2.rectangle(frame, (x, y), (x+w, y+h), (255, 255, 255), 1)
```

```
# 6. 在畫面上加上文字標籤 (進階挑戰要求)
# 參數：圖片，文字，位置，字體，大小，顏色(BGR)，厚度
cv2.putText(frame, "Privacy Mode: ON", (20, 50),
            cv2.FONT_HERSHEY_SIMPLEX, 1, (0, 255, 0), 2)

# 7. 顯示結果視窗
cv2.imshow('Smart Privacy Guard', frame)

# 8. 偵測鍵盤按鍵，若按下 'q' 則離開迴圈
if cv2.waitKey(1) & 0xFF == ord('q'):
    break

# 9. 釋放資源與關閉視窗
cap.release()
cv2.destroyAllWindows()
```

上堂我們學習了電腦視覺中的其中一種技術 — 臉部識別。
今堂我們將會學習另一種技術 — 物件追蹤。

2. 什麼是物件追蹤 (Object Tracking)

物件追蹤是電腦視覺的一種技術，用來在影片中持續追蹤某個物件的位置。

例如：

- 追蹤跑步中的運動員
- 追蹤路上的車輛
- 追蹤鏡頭中的人物

物件追蹤廣泛應用於：

- 視頻監控
- 自動駕駛
- 運動分析

項目	物件追蹤 Object Tracking	物件偵測 Object Detection
目標	在影片中追蹤特定物體的移動	識別並定位圖像或影片中的物體
原理	分析多幀畫面，連續追蹤物件位置	逐幀分析，每一幀重新找出物件
常見方法	GOTURN、Meanshift、CAMshift、光流法	YOLO、Faster R-CNN 等

物件追蹤的演算法

OpenCV 提供了多種物件追蹤演算法：

演算法	說明
BOOSTING	元老級追蹤器，速度較慢，而且不太準確。
MIL	比 BOOSTING 更準確，但仍然不算很好。
GOTURN	需要配合深度學習模型使用。
TLD	速度普通，精準度普通。
MEDIANFLOW	對快速移動或跳動物件較不準確。
KCF	速度快，表現不錯，但遮擋時較易失敗。
MOSSE	速度最快，但準確度比 KCF 和 CSRT 稍差。
CSRT	準確度很好，但速度比 KCF 慢。

3. 程式流程：物件追蹤六部曲

要在影片中追蹤一個物件，通常會有以下步驟：

1. **導入模組**: import cv2 及 `pip install opencv-contrib-python`
2. **選擇追蹤演算法**: 從上面提到的演算法中選擇一個。
3. **讀取影片第一幀**: 不是播放整段影片，只顯示影片第一幀，以方便下一步。
4. **手動圈選要追蹤的目標 (ROI)**: 在視窗中，利用滑鼠圈出目標。
5. **初始化追蹤器**: 令演算法知道要追蹤的目標。
6. **持續更新追蹤結果並畫出方框**: 在偵測到的位置畫出長方形。

程式

```
# 1. 導入函數庫
import cv2

# 建立一個變數儲存揀選的演算法
tracker_type = 'BOOSTING'

# 根據選擇的演算法類型初始化追蹤器
if tracker_type == 'BOOSTING':
    tracker = cv2.legacy.TrackerBoosting_create()
elif tracker_type == 'MIL':
    tracker = cv2.legacy.TrackerMIL_create()
elif tracker_type == 'KCF':
    tracker = cv2.legacy.TrackerKCF_create()
elif tracker_type == 'TLD':
    tracker = cv2.legacy.TrackerTLD_create()
elif tracker_type == 'MEDIANFLOW':
    tracker = cv2.legacy.TrackerMedianFlow_create()
elif tracker_type == 'MOSSE':
    tracker = cv2.legacy.TrackerMOSSE_create()
elif tracker_type == 'CSRT':
    tracker = cv2.legacy.TrackerCSRT_create()

# 開啟影片檔案
video = cv2.VideoCapture('race.mp4')
isTrue, frame = video.read()

# 在第一幀圈選要追蹤的物件 (ROI)
bbox = cv2.selectROI(frame) # region of interest
```

```
# 初始化追蹤器
ok = tracker.init(frame, bbox)

# 設定矩形框顏色 (紅色)
colors = (0, 0, 255) # RGB -> BGR

# 開始一個迴圈以追蹤
while True:
    isTrue, frame = video.read()
    if not isTrue:
        break

    # 更新追蹤器並在影片上畫出邊界框
    ok, bbox = tracker.update(frame)

    # 如果成功追蹤到物件
    if ok == True:
        (x, y, w, h) = [int(v) for v in bbox]
        cv2.rectangle(frame, (x, y), (x + w, y + h), colors, 2)

    # 在畫面上顯示演算法類型 (左上角)
    cv2.putText(frame, tracker_type, (100, 20),
                cv2.FONT_HERSHEY_SIMPLEX, .75, (0, 0, 255))

    # 顯示結果影像
    cv2.imshow('Tracking', frame)

    # 按下 ESC 鍵退出
    if cv2.waitKey(1) & 0xFF == 27: # esc
        break

video.release()
cv2.destroyAllWindows()
```

任務挑戰**任務一：編寫一個 Python 程式《演算法大比拼》**

請使用以下追蹤演算法測試同一段影片：

- BOOSTING
- MIL
- KCF
- TLD
- MEDIANFLOW
- MOSSE
- CSRT

要求：

- 使用同一條影片
- 每次只更改追蹤演算法
- 比較哪一種最準確
- 然後填寫下表
- 並輸入至 MS Forms 之中

演算法名稱	執行速度 (快/一般/慢)	追蹤穩定度 (1 - 5 分)	遇到遮擋時的表現 (跟丟 / 跟緊)
BOOSTING			
MIL			
KCF			
TLD			
MEDIANFLOW			
MOSSE			
CSRT			

任務二：編寫一個 Python 程式《鏡頭追蹤器》：

- 使用 cv2.VideoCapture(0) 開啟鏡頭
- 先手動圈出要追蹤的物件並按下
- 即時追蹤該物件
- 在物件外畫上紫色方框
- 按下空白鍵結束程式。

(提交 Task2_3X99.py)